

Al-Nahrain University
College of Information Engineering
Department of Computer Networks Engineering



Report on

Software-Defined Networking (SDN)

By:
Baiqr Ahmed Razzaq

Supervised By:
Dr. Khalid W. Hameed

Introduction

In traditional networks, each device (such as a router or switch) performs its own networking functions independently, including forwarding and re-forwarding data across the network. As the number of devices increases, managing and controlling the network becomes increasingly difficult.

Software-Defined Networking (SDN) addresses this problem by separating network functions into two distinct layers:

> **Control Plane:** responsible for directing and controlling the network
> **Data Plane:** consists of the network devices (switches/routers) and is responsible for forwarding data

This separation allows the network to be managed through software and centralized tools, rather than configuring each device manually and individually, a process that would otherwise take a long time.

SDN Architecture

The SDN architecture consists of three main layers:
Application Layer: Includes applications responsible for network management, policy enforcement, and security services.
Control Layer: Provides centralized network control, maintains a global view of the network, and connects applications with the underlying infrastructure.
Infrastructure Layer: Consists of network devices responsible for forwarding traffic according to instructions received from the control layer.

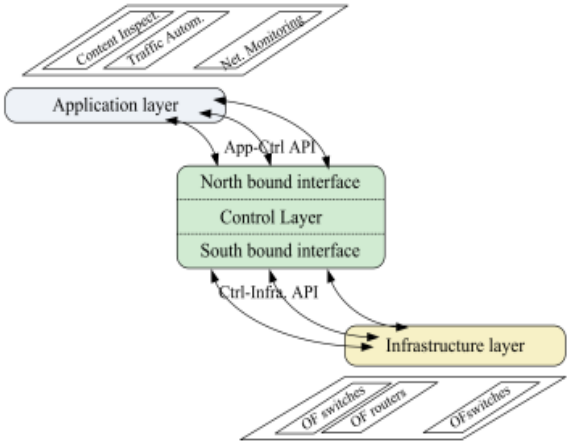


Fig. 1. The three planes/layers in the SDN architecture.

Communication between these layers is achieved through two main interfaces. The **Northbound API (App-Ctrl API)** connects the application layer to the control layer, while the **Southbound API (Ctrl-Infra API)** connects the control layer to the infrastructure layer. OpenFlow is an example of a protocol used for communication between the control and infrastructure layers.

Operation of SDN

When a packet enters a switch, it is checked against the switch's flow table. If a matching rule is found, the switch applies the associated action directly for example, forwarding the packet or dropping it. If no match is found,
the switch typically follows a default rule that sends the packet to the controller as a Packet-In message.
The controller then decides how the packet should be handled and installs
a new rule on the switch so that future packets of the same type
are processed locally
without involving the controller again.

SDN/OpenFlow

In an SDN/OpenFlow architecture, there are two main elements: the controllers and the forwarding devices. A forwarding device is a hardware or software element specialized in packet forwarding, while a controller is a software stack often described as the network brain, running on commodity hardware. OpenFlow defines how these two elements communicate, letting the controller install, update, and remove forwarding rules on switches from different vendors through a single, open interface.

Fundamental Principles of SDN

SDN is based on four main principles:

1. **Separation of Control and Data Planes:** SDN separates the control functions from network devices, which become simple forwarding elements responsible for handling data packets.
2. **Flow-Based Forwarding:** Unlike traditional destination-based forwarding, SDN makes forwarding decisions based on flows. A flow is defined by specific packet fields used for matching, along with a set of actions that determine how packets are handled. Packets belonging to the same flow receive the same forwarding policies.
3. **Centralized Control:** The control logic is moved to an external entity called the SDN Controller or Network Operating System (NOS). It provides the necessary resources and maintains a logically centralized view of the network to manage and program forwarding devices.
4. **Network Programmability:** SDN allows the network to be programmed through software applications running on top of the NOS. These applications interact with the underlying network devices, making it possible to control and manage their operations through software.

SDN Applications

SDN has been applied across several domains, including traffic engineering, mobility and wireless networks, measurement and monitoring, data center networking, and security and dependability. In the security domain specifically, SDN's centralized and programmable nature has been used to build applications such as access control, firewalling, and the detection and mitigation of DDoS attacks. At the same time, because this centralization concentrates control in one place, part of the research in this area has also focused on securing the SDN controller itself.

Benefits

By separating the network's control logic from the underlying switches and promoting logical centralization, SDN simplifies network management and makes it easier to introduce new abstractions and policies. This programmability allows network behavior to be changed through software rather than through manual, device-by-device configuration, which also facilitates the evolution of the network over time.

Challenges

A single controller may be sufficient to manage a small network, but it represents a single point of failure. If the controller becomes overloaded or unavailable, the entire network it manages can be affected. This is one of the main trade-offs of SDN's centralized design, and it has motivated research into distributing control across multiple controllers to reduce this risk.

In summary, SDN redefines network management by separating control from forwarding and introducing centralized, programmable control. While this design brings significant benefits in flexibility and ease of management, it also introduces centralization as a potential weak point that must be carefully addressed.

References

Software-Defined Networking: A Comprehensive Survey, Diego Kreutz